



МИНИСТЕРСТВО СЕЛЬСКОГО ХОЗЯЙСТВА РОССИЙСКОЙ ФЕДЕРАЦИИ
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ БЮДЖЕТНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО
ОБРАЗОВАНИЯ

«РОССИЙСКИЙ ГОСУДАРСТВЕННЫЙ АГРАРНЫЙ УНИВЕРСИТЕТ –
МСХА имени К. А. ТИМИРЯЗЕВА

(ФГБОУ ВО РГАУ – МСХА имени К. А. Тимирязева)

Институт экономики и управления

Кафедра статистики и кибернетики

Направление: 09.03.02 Информационные системы и технологии

Направленность:

Большие данные и машинное обучение

КУРСОВОЙ ПРОЕКТ

на тему: «Проектирование и разработка информационной системы для имитационного моделирования логистических процессов в агропромышленном комплексе».

Выполнил:

Студент 3 курса группы Д-ЭЗ11

Недосекина Е.С.

Дата регистрации КП

на кафедре: _____

Допущен к защите

Руководитель:

учёная степень, учёное звание, ФИО

Члены комиссии:

учёная степень, учёное звание, ФИО

подпись

учёная степень, учёное звание, ФИО

подпись

учёная степень, учёное звание, ФИО

подпись

Оценка _____

Дата защиты _____

Москва 2026

Содержание

Введение	3
1 Глава 1. Теоретические основы имитационного моделирования логистических процессов в агропромышленном комплексе	5
1.1. Понятие и сущность имитационного моделирования	5
1.2. Виды имитационных моделей и их применение в логистике . .	6
1.3. Особенности логистических процессов в агропромышленном комплексе	7
2 Глава 2. Проектирование и разработка информационной системы имитационного моделирования на базе Django	9
2.1. Обоснование выбора технологий: Python, Django, SQLite	9
2.2. Проектирование архитектуры системы	10
2.3. Программная реализация основных компонентов системы . . .	13
2.4. Структура проекта и организация программного кода	16
2.5. Разработка веб-интерфейса информационной системы	20
2.6. Реализация имитационного ядра и тестирование системы	24
Заключение	26
Список использованных источников	28
Приложения	29

Введение

Современное сельскохозяйственное производство характеризуется высокой степенью механизации и значительной численностью персонала, задействованного в полевых и животноводческих работах. Эффективность труда на предприятиях агропромышленного комплекса напрямую зависит не только от технической оснащённости, но и от своевременного обеспечения работников средствами индивидуальной защиты. Спецодежда является обязательным элементом охраны труда, а её отсутствие или несвоевременная замена могут привести к нарушениям техники безопасности, простоям и снижению производительности.

Несмотря на важность данного вопроса, многие сельскохозяйственные предприятия до сих пор ведут учёт спецодежды с использованием бумажных журналов либо электронных таблиц. Такой подход позволяет фиксировать факт выдачи, но не даёт возможности прогнозировать будущие потребности. Кладовщик узнаёт о нехватке того или иного предмета лишь в момент обращения работника, когда замена требуется немедленно. В условиях сезонных полевых работ, когда нагрузка на персонал и, соответственно, на одежду возрастает кратно, подобная ситуация способна привести к серьёзным сбоям в производственном процессе.

Одним из способов решения данной проблемы является применение методов имитационного моделирования. Имитационная модель позволяет воспроизвести поведение реальной системы в ускоренном масштабе времени, учитывая нормативные сроки службы предметов и случайные факторы, влияющие на их износ. В отличие от обычного учёта, который лишь отражает текущее состояние, имитация даёт возможность заглянуть вперёд и заблаговременно определить момент возникновения дефицита.

Целью работы является проектирование и разработка информационной системы для имитационного моделирования логистических процессов обеспечения спецодеждой в агропромышленном комплексе. Система реализуется в виде веб-сайта на базе фреймворка Django, что обеспечивает доступность с любого устройства через браузер и не требует установки дополнительного программного обеспечения на рабочих местах.

Для достижения поставленной цели необходимо решить следующие задачи:

- 1) Изучить теоретические основы имитационного моделирования и определить возможности его применения в сфере учёта спецодежды;
- 2) Обосновать выбор инструментов разработки: язык программирования Python, фреймворк Django, систему управления базами данных SQLite, среду PyCharm;
- 3) Спроектировать структуру базы данных, включающую сведения о работниках, номенклатуре спецодежды и параметрах, необходимых для имитации износа;

- 4) Реализовать веб-интерфейс, обеспечивающий учёт работников, складских остатков, фиксацию выдачи одежды и отображение текущего состояния предметов;
- 5) Разработать алгоритм имитационного моделирования, рассчитывающий степень износа каждой единицы одежды за заданный период с учётом нормативных сроков и стохастической составляющей.

Объектом исследования выступают логистические процессы обеспечения спецодеждой на сельскохозяйственном предприятии. **Предметом исследования** является имитационная модель оборота средств индивидуальной защиты, реализованная в виде веб-ориентированной информационной системы.

Практическая значимость работы заключается в создании готового программного продукта, который может быть использован на малых и средних предприятиях агропромышленного комплекса для автоматизации учёта спецодежды и прогнозирования сроков её замены.

Глава 1. Теоретические основы имитационного моделирования логистических процессов в агропромышленном комплексе

1.1. Понятие и сущность имитационного моделирования

Имитационное моделирование представляет собой метод исследования, при котором изучаемая система заменяется её упрощённой копией, то есть моделью, способной воспроизводить поведение оригинала с достаточной для практических целей точностью. В отличие от аналитических методов, опирающихся на строгие математические формулы, имитация позволяет экспериментировать с моделью во времени, наблюдая за её реакцией на различные входные воздействия.

Термин «имитационное моделирование» ввёл в научный оборот американский учёный Роберт Шеннон в середине 1970-х годов. Он определил его как процесс конструирования модели реальной системы и постановки экспериментов на этой модели с целью либо понять поведение системы, либо оценить различные стратегии, обеспечивающие её функционирование. С тех пор данное направление получило широкое развитие и применяется в самых разных областях: от промышленного производства до здравоохранения, от логистики до военного дела.

Ключевая особенность имитационного подхода состоит в том, что он не требует построения сложных математических зависимостей. Если аналитическая модель пытается описать систему набором уравнений, которые зачастую невозможно решить для реальных условий, то имитационная модель просто повторяет логику работы системы шаг за шагом. Это позволяет учитывать случайные факторы, нелинейные связи и другие особенности, делающие аналитическое решение невозможным.

Процесс имитационного моделирования принято разделять на несколько последовательных этапов. Первый этап представляет собой формулировку проблемы и постановку задачи. Второй этап заключается в построении концептуальной модели, то есть словесного или графического описания структуры системы и логики её функционирования. Третий этап охватывает формализацию, перевод концептуальной модели на язык, понятный компьютеру: в виде алгоритмов, блок-схем или программного кода. Четвёртый этап включает програм-

мирование модели и её отладку. Пятый этап посвящён проведению экспериментов с моделью, варьированию входных параметров и накоплению результатов. Заключительный, шестой этап предполагает анализ полученных данных и формулировку выводов, которые могут быть применены к реальной системе.

Применение имитационного моделирования особенно оправдано в тех случаях, когда проведение реальных экспериментов невозможно или нецелесообразно. Например, нельзя остановить работу сельскохозяйственного предприятия на месяц, чтобы посмотреть, что произойдёт со складскими запасами. Гораздо проще «проиграть» этот месяц на модели за несколько секунд, получив при этом достаточно точный прогноз. Кроме того, имитация позволяет многократно повторять один и тот же сценарий с разными исходными данными, сравнивая результаты и выбирая наилучшую стратегию управления.

Таким образом, имитационное моделирование является мощным инструментом анализа сложных систем, позволяющим учитывать множество факторов одновременно и получать практически значимые результаты без вмешательства в работу реального предприятия.

1.2. Виды имитационных моделей и их применение в логистике

Современная теория выделяет три основных подхода к построению имитационных моделей: системную динамику, дискретно-событийное моделирование и агентное моделирование. Каждый из этих подходов имеет свои особенности, а выбор конкретного метода зависит от характера решаемой задачи.

Системная динамика, разработанная Джейм Форрестером в 1950-х годах, рассматривает систему на высоком уровне абстракции. Данный подход оперирует понятиями потоков и накопителей. Потоки представляют собой движение ресурсов между накопителями, а накопители характеризуют текущее состояние системы. Например, склад можно представить как накопитель, в который поступает поток новых предметов одежды и из которого уходит поток выдаваемых вещей. Системная динамика хорошо подходит для анализа общих тенденций, но не позволяет отслеживать поведение отдельных объектов.

Дискретно-событийное моделирование, напротив, фокусируется на конкретных событиях, происходящих в системе в определённые моменты времени. Событием может быть получение товара на склад, выдача одежды работнику или списание изношенного предмета. Между событиями состояние системы считается неизменным. Такой подход удобен для моделирования производственных линий, очередей, складских операций. Он позволяет детально отслеживать каждую операцию, но требует значительных вычислительных ресурсов при большом количестве событий.

Агентное моделирование представляет систему как совокупность самостоя-

тельных объектов, или агентов. Каждый агент обладает набором свойств и правил поведения, может взаимодействовать с другими агентами и со средой. Поведение всей системы складывается из индивидуальных действий множества агентов. В логистике агентами могут выступать транспортные средства, единицы товара или отдельные работники. Данный подход особенно эффективен, когда объекты системы имеют индивидуальные характеристики, влияющие на общий результат.

На практике перечисленные подходы часто комбинируются. В рамках настоящей работы используется сочетание дискретно-событийного и агентного моделирования. Время в модели изменяется дискретно: один шаг имитации соответствует одному месяцу работы. Единицы выданной спецодежды выступают в роли агентов, каждый из которых характеризуется собственным нормативным сроком службы и текущим уровнем износа. Событиями являются выдача одежды, ежемесячное изменение износа и достижение критического состояния, требующего замены.

1.3. Особенности логистических процессов в агропромышленном комплексе

Логистика в агропромышленном комплексе обладает рядом отличительных черт, которые выделяют её среди других отраслей экономики. Понимание этих особенностей необходимо для корректной постановки задачи моделирования и разработки эффективной информационной системы.

Первой и наиболее значимой особенностью является разнообразие профессий и, как следствие, широкая номенклатура используемой спецодежды. На типичном сельскохозяйственном предприятии одновременно трудятся механизаторы, животноводы, операторы машинного доения, работники зернотоков и складов. Каждой категории работников полагается свой набор средств индивидуальной защиты. Комбинезоны, сапоги, перчатки, халаты и головные уборы различаются по конструкции, материалам и, что особенно важно для учёта, по нормативным срокам эксплуатации. Это многообразие существенно усложняет задачу кладовщика, вынужденного отслеживать движение десятков позиций с разными нормативами одновременно.

Второй особенностью является различие в нормативных сроках службы разных видов спецодежды. Согласно типовым нормам, утверждённым Министерством труда, срок носки перчаток может составлять всего 30 дней, тогда как зимняя куртка выдаётся на два-три года. Кладовщику необходимо помнить, что одни позиции требуют замены ежемесячно, а другие служат годами, и учитывать это при формировании заявок на закупку. Без автоматизированной системы легко упустить момент, когда подходит срок замены той или иной позиции.

Третья особенность связана с индивидуальным характером износа. Даже в

пределах одного норматива фактический срок службы одежды у разных работников может заметно отличаться. Это зависит от аккуратности сотрудника, интенсивности его труда, конкретных условий на рабочем месте. Один механизатор может носить комбинезон весь положенный срок, а у другого он придёт в негодность значительно раньше. Имитационная модель, включающая случайную составляющую износа, позволяет приблизиться к реальной картине, в отличие от жёсткого нормативного подхода, который предполагает, что все вещи изнашиваются строго по графику.

Четвёртой особенностью является нормативное регулирование. Обеспечение работников средствами индивидуальной защиты регламентируется Трудовым кодексом Российской Федерации и отраслевыми приказами Министерства труда. Типовые нормы устанавливают конкретные сроки носки для каждой профессии и каждого вида спецодежды. Работодатель обязан соблюдать эти нормы, а их нарушение влечёт административную ответственность. Следовательно, логистическая система должна не только учитывать текущие остатки на складе, но и обеспечивать своевременную замену изношенных предметов в соответствии с установленными сроками.

Перечисленные особенности делают задачу управления снабжением спецодеждой нетривиальной. Обычные методы учёта, основанные на ручной фиксации текущих остатков, не учитывают ни разницы в нормативах, ни индивидуальных отклонений в сроках службы. Здесь и становится востребованным имитационное моделирование, позволяющее спрогнозировать развитие ситуации на несколько шагов вперёд и заранее определить момент, когда та или иная позиция потребует замены.

Глава 2. Проектирование и разработка информационной системы имитационного моделирования на базе Django

2.1. Обоснование выбора технологий: Python, Django, SQLite

Разработка любого программного продукта начинается с выбора инструментов, которые будут использоваться на всех этапах создания системы. Применительно к данной работе выбор технологий определялся следующими критериями: доступность для освоения, наличие готовых компонентов для решения типовых задач веб-разработки, соответствие учебному характеру проекта и возможность быстрого получения работающего прототипа.

В качестве языка программирования был выбран Python. Он отличается простым и читаемым синтаксисом, что особенно важно в учебных проектах. Python имеет огромное количество библиотек для решения самых разных задач, а его активное сообщество позволяет быстро найти ответы на возникающие вопросы.

В качестве веб-фреймворка использован Django. Данный фреймворк предоставляет готовую архитектуру для создания веб-приложений, включая объектно-реляционное отображение для работы с базой данных, систему маршрутизации, шаблонизатор и встроенную административную панель. Это позволяет сосредоточиться на бизнес-логике приложения, не тратя время на реализацию типовых элементов.

В качестве системы управления базами данных использована SQLite. Данная СУБД входит в стандартную поставку Python и Django, не требует отдельной установки и настройки сервера, а вся база данных хранится в одном файле. При этом переход на промышленную СУБД, такую как PostgreSQL, в случае необходимости потребует лишь изменения нескольких строк в настройках проекта без модификации кода моделей или представлений.

Среда разработки PyCharm предоставляет удобный редактор кода с автодополнением, встроенный терминал, инструменты для работы с системой контроля версий и отладчик.

Для оформления пользовательского интерфейса применён фреймворк Bootstrap 5, подключаемый через CDN. Это позволило получить аккуратный внешний вид страниц с адаптивной вёрсткой без написания собственных CSS-стилей.

Таким образом, выбранный стек технологий (Python, Django, SQLite, PyCharm, Bootstrap) обеспечивает баланс между простотой разработки и функциональными возможностями.

2.2. Проектирование архитектуры системы

В Django модели данных описываются в виде классов Python, а фреймворк автоматически создаёт соответствующие таблицы в базе. В разработанной системе выделены три основные сущности в папке `models.py`: работник, спецодежда и запись о выдаче.

Модель `Worker` (работник) содержит два поля (Рис. 1): `full_name` (ФИО) и `position` (должность). Этого достаточно для идентификации сотрудника и определения набора полагающейся ему одежды в соответствии с типовыми нормами для данной должности.

```
class Worker(models.Model): 8 usages
    """Работник (доярка, тракторист и т.д.)"""
    full_name = models.CharField(max_length=100, verbose_name="ФИО")
    position = models.CharField(max_length=100, verbose_name="Должность")
    def __str__(self):
        return f"{self.full_name} ({self.position})"
    class Meta:
        verbose_name = "Работник"
        verbose_name_plural = "Работники"
```

Рисунок 1 — Модель `Worker`

Модель `SpecOdejda` (спецодежда) описывает номенклатуру склада (Рис. 2). Помимо названия и размера, важнейшим полем является `norm_days` (нормативный срок носки в днях). Именно это значение используется в алгоритме имитации для расчёта скорости износа. Поле `quantity_in_stock` отражает текущий остаток на складе и автоматически уменьшается при каждой выдаче.

```

class SpecOdejda(models.Model): 8 usages
    """Номенклатура спецодежды на складе"""
    TYPES = [
        ('upper', 'Верхняя одежда'),
        ('shoes', 'Обувь'),
        ('gloves', 'Перчатки'),
        ('head', 'Головной убор'),
        ('other', 'Прочее'),
    ]

    name = models.CharField(max_length=150, verbose_name="Наименование")
    type = models.CharField(max_length=20, choices=TYPES, verbose_name="Тип")
    size = models.CharField(max_length=10, verbose_name="Размер")
    norm_days = models.IntegerField(verbose_name="Срок носки по нормативу (дней)")
    quantity_in_stock = models.IntegerField(default=0, verbose_name="Количество на складе")

    def __str__(self):
        return f"{self.name} (разм. {self.size})"

    class Meta:
        verbose_name = "Единица спецодежды"
        verbose_name_plural = "Спецодежда"

```

Рисунок 2 — Модель SpecOdejda

Модель IssuedItem (запись о выдаче) связывает работника с конкретным предметом одежды и хранит параметры, необходимые для имитации (Рис. 3). Поле wear_level показывает текущий процент износа (от 0 для новой вещи до 100 для полностью изношенной). Поле status принимает одно из трёх значений: «Выдана», «Изношена» или «Списана». При создании записи автоматически фиксируется дата выдачи.

```

class IssuedItem(models.Model): 10 usages
    """Журнал выдачи: какая вещь кому выдана и её текущее состояние"""
    STATUSES = [
        ('issued', 'Выдана'),
        ('worn_out', 'Изношена'),
        ('written_off', 'Списана'),
    ]

    worker = models.ForeignKey(Worker, on_delete=models.CASCADE, verbose_name="Работник")
    spec_odejda = models.ForeignKey(SpecOdejda, on_delete=models.CASCADE, verbose_name="Спецодежда")
    issue_date = models.DateField(auto_now_add=True, verbose_name="Дата выдачи")
    wear_level = models.IntegerField(default=0, verbose_name="Износ (%)")
    status = models.CharField(max_length=20, choices=STATUSES, default='issued', verbose_name="Статус")

    def __str__(self):
        return f"{self.spec_odejda.name} → {self.worker.full_name}"

    class Meta:
        verbose_name = "Запись о выдаче"
        verbose_name_plural = "Журнал выдачи"

```

Рисунок 3 — Модель IssuedItem

Связи между моделями организованы по принципу «один ко многим»: один работник может иметь несколько записей о выданной одежде, один вид спец-одежды может быть выдан многократно разным работникам.

Система предоставляет следующие функции, доступные через веб-интерфейс: просмотр сводной информации на главной странице (количество работников, видов одежды, выданных и изношенных предметов); просмотр списка работников с отображением выданной и изношенной одежды; просмотр складских остатков с цветовой индикацией; выдача одежды работнику через форму; запуск имитации одного месяца работы; просмотр списка изношенной одежды, требующей замены. Доступ к административной панели Django позволяет управлять всеми данными напрямую.

Ядром системы, реализующим имитационное моделирование, является функция `run_simulation`. На первом шаге из базы данных выбираются все записи со статусом «Выдана». Для каждой найденной записи определяется нормативный срок службы данного вида одежды. Затем вычисляется базовый износ за один месяц по формуле: `base_wear` равен частному от деления 30 на `norm_days`, умноженному на 100 процентов. К базовому износу применяется случайный коэффициент в диапазоне от 0,7 до 1,3, что позволяет имитировать индивидуальные различия в условиях эксплуатации. Полученное значение прибавляется к текущему уровню износа `wear_level`. Если после этого `wear_level` достигает или превышает 100 процентов, статус записи меняется на «Изношена». Результаты сохраняются в базе данных. Пользователь видит информационное сообщение о количестве обработанных записей и количестве предметов, изношенных пол-

ностью за данный шаг имитации.

На рисунке 4 приведён программный код функции `run_simulation`, реализующей один шаг имитационного моделирования. Функция последовательно обрабатывает все активные записи о выданной одежде, рассчитывает износ за месяц с учётом норматива и случайного фактора, а при достижении критического уровня переводит предмет в категорию изношенных.

```
def run_simulation(request): 1 usage
    """
    Имитация одного месяца работы.
    Каждая выданная вещь изнашивается на случайную величину (20-40%).
    """

    issued_items = IssuedItem.objects.filter(status='issued')

    for item in issued_items:
        # Имитируем износ: добавляем случайные 20-40%
        import random
        wear_increase = random.randint(a=20, b=40)
        item.wear_level += wear_increase

        # Если износ достиг или превысил 100% — вещь изношена
        if item.wear_level >= 100:
            item.wear_level = 100
            item.status = 'worn_out'

        item.save()

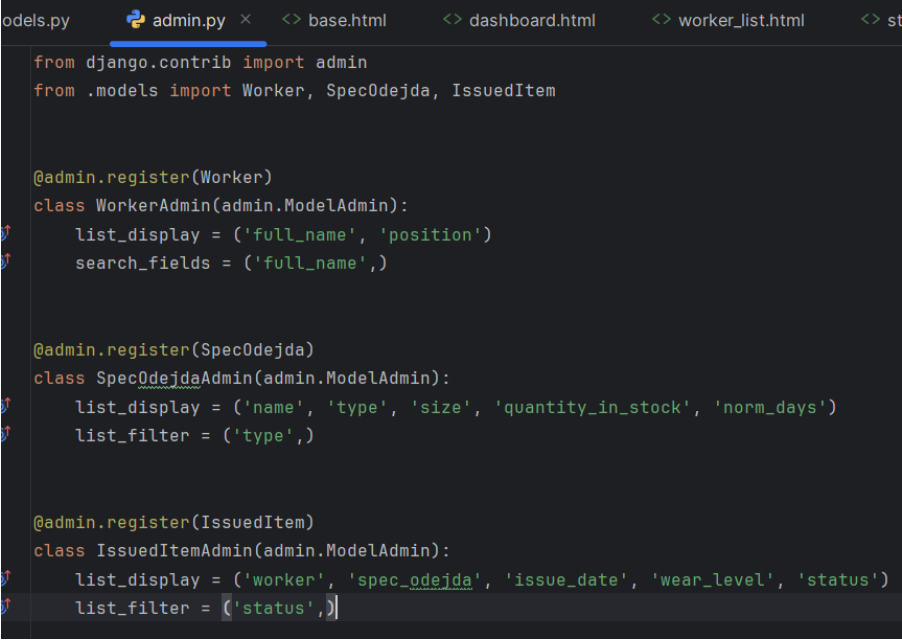
    messages.info(request, message=f'⚙ Имитация завершена! Обработано {issued_items.count()} записей.')
    return redirect('dashboard')
```

Рисунок 4 — Функция `run_simulation` из папки `views.py`

2.3. Программная реализация основных компонентов системы

Одним из преимуществ Django является автоматически генерируемая административная панель. Чтобы сделать её удобной для работы, необходимо зарегистрировать модели и настроить отображение полей. В файле `admin.py` для каждой модели указывается список полей, выводимых в таблице, а также настраиваются фильтры и поиск.

На рисунке 5 представлен код настройки административной панели. Для каждой модели указаны поля, отображаемые в виде таблицы, что упрощает навигацию по данным. Добавление фильтров позволяет быстро находить записи по типу одежды или статусу выдачи.



```

models.py  admin.py  base.html  dashboard.html  worker_list.html  st
from django.contrib import admin
from .models import Worker, SpecOdejda, IssuedItem

@admin.register(Worker)
class WorkerAdmin(admin.ModelAdmin):
    list_display = ('full_name', 'position')
    search_fields = ('full_name',)

@admin.register(SpecOdejda)
class SpecOdejdaAdmin(admin.ModelAdmin):
    list_display = ('name', 'type', 'size', 'quantity_in_stock', 'norm_days')
    list_filter = ('type',)

@admin.register(IssuedItem)
class IssuedItemAdmin(admin.ModelAdmin):
    list_display = ('worker', 'spec_odejda', 'issue_date', 'wear_level', 'status')
    list_filter = ('status',)

```

Рисунок 5 — Код административной панели

Представления являются связующим звеном между моделями данных и шаблонами страниц. Они получают запрос от пользователя, при необходимости извлекают или изменяют данные в базе и передают результат в шаблон для отображения. Рассмотрим устройство двух ключевых представлений: выдачи одежды и имитации износа.

Функция `issue_form` обрабатывает как GET-запросы (отображение формы), так и POST-запросы (обработка отправленных данных). При получении POST-запроса функция извлекает идентификаторы выбранного работника и предмета одежды. Далее выполняется проверка наличия предмета на складе. Если количество больше нуля, создаётся новая запись в журнале выдачи с нулевым уровнем износа, а складской остаток уменьшается на единицу. Пользователь получает подтверждающее сообщение. Если предмета нет в наличии, выводится сообщение об ошибке. При GET-запросе формируется список работников и список доступной одежды для отображения в выпадающих списках формы.

На рисунке 6 показан код функции `issue_form`, отвечающей за выдачу спец-одежды. Функция демонстрирует типичный для Django подход к обработке форм: разделение логики для GET и POST запросов, взаимодействие с базой данных через ORM и использование механизма сообщений для информирования пользователя.

```

def issue_form(request):
    if request.method == 'POST':
        worker_id = request.POST.get('worker_id')
        item_id = request.POST.get('item_id')
        worker = Worker.objects.get(id=worker_id)
        item = SpecOdejda.objects.get(id=item_id)
        # Проверяем, есть ли на складе
        if item.quantity_in_stock > 0:
            # Создаём запись о выдаче
            IssuedItem.objects.create(
                worker=worker,
                spec_odejda=item,
                wear_level=0, # Новая вещь — износ 0%
            )
            # Уменьшаем количество на складе
            item.quantity_in_stock -= 1
            item.save()
            messages.success(request, message=f'{item.name} выдано работнику {worker.full_name}')
        else:
            messages.error(request, message=f'{item.name} отсутствует на складе!')

    return redirect('issue_form')

# GET-запрос — показываем форму
workers = Worker.objects.all()
# Показываем только то, что есть в наличии
items = SpecOdejda.objects.filter(quantity_in_stock__gt=0)
recent_issues = IssuedItem.objects.all().order_by('-issue_date')[:10]
context = {
    'workers': workers,
    'items': items,
    'recent_issues': recent_issues,
}

```

Рисунок 6 — Функция `issue_form`

Для связи адресов страниц с функциями представлений используется файл `urls.py`. Каждому URL сопоставляется имя, которое затем используется в шаблонах для генерации ссылок.

На рисунке 7 приведён файл маршрутизации приложения. Каждому пути соответствует своя функция-обработчик. Именованные маршруты позволяют ссылаться на страницы по символьным именам, а не по жёстко заданным адресам, что упрощает поддержку проекта.

```

from django.urls import path
from . import views

urlpatterns = [
    path('', views.dashboard, name='dashboard'),
    path('workers/', views.worker_list, name='worker_list'),
    path('stock/', views.stock_list, name='stock_list'),
    path('issue/', views.issue_form, name='issue_form'),
    path('simulate/', views.run_simulation, name='run_simulation'),
    path('worn/', views.worn_items_list, name='worn_items'),
]

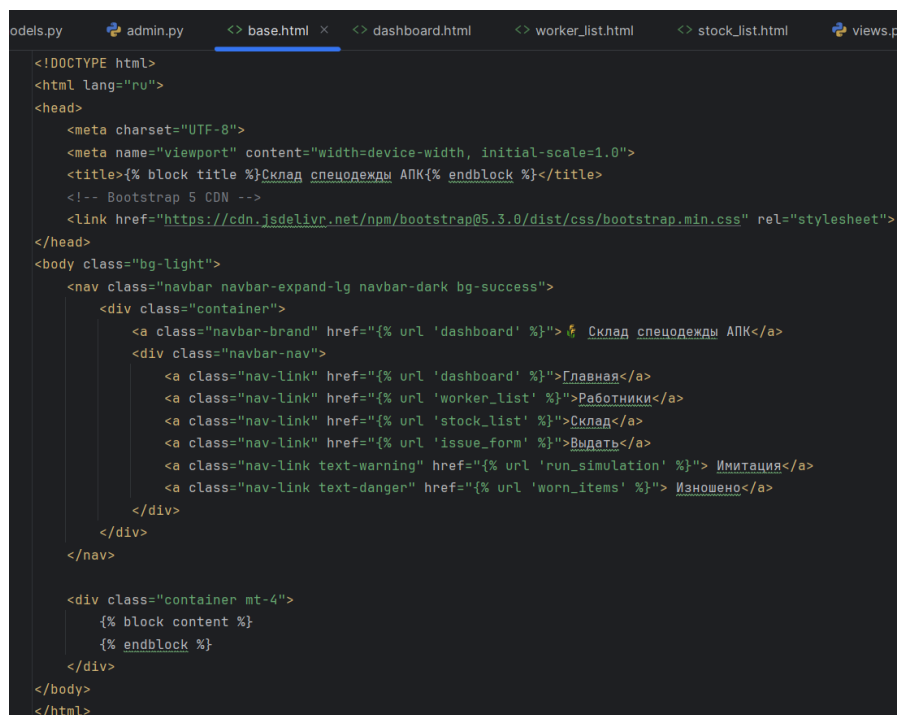
```

Рисунок 7 — Файл маршрутизации приложения

Все страницы системы построены на основе единого базового шаблона `base.html`.

который содержит навигационную панель и подключает фреймворк Bootstrap. Остальные шаблоны наследуются от базового и определяют только содержимое основной области страницы. Такой подход исключает дублирование кода и обеспечивает единообразие оформления.

На рисунке 8 показан базовый шаблон base.html. В нём определена общая структура страниц, подключены стили Bootstrap и создана навигационная панель. Блок content является местом для вставки содержимого, определяемого в дочерних шаблонах.



```
<!DOCTYPE html>
<html lang="ru">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>{% block title %}Склад спецодежды АПК{% endblock %}</title>
  <!-- Bootstrap 5 CDN -->
  <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.0/dist/css/bootstrap.min.css" rel="stylesheet">
</head>
<body class="bg-light">
  <nav class="navbar navbar-expand-lg navbar-dark bg-success">
    <div class="container">
      <a class="navbar-brand" href="{% url 'dashboard' %}">🚚 Склад спецодежды АПК</a>
      <div class="navbar-nav">
        <a class="nav-link" href="{% url 'dashboard' %}">Главная</a>
        <a class="nav-link" href="{% url 'worker_list' %}">Работники</a>
        <a class="nav-link" href="{% url 'stock_list' %}">Склад</a>
        <a class="nav-link" href="{% url 'issue_form' %}">Выдать</a>
        <a class="nav-link text-warning" href="{% url 'run_simulation' %}">Имитация</a>
        <a class="nav-link text-danger" href="{% url 'worn_items' %}">Изношено</a>
      </div>
    </div>
  </nav>

  <div class="container mt-4">
    {% block content %}
    {% endblock %}
  </div>
</body>
</html>
```

Рисунок 8 — Файл base.html

2.4. Структура проекта и организация программного кода

Любой проект на Django имеет определённую файловую структуру, которая закладывается автоматически при создании проекта и приложений. Понимание этой структуры необходимо для дальнейшей разработки и сопровождения системы. В данном параграфе рассмотрено назначение каждого элемента проекта и пояснено, какие файлы были созданы или изменены в ходе работы.

Проект Django состоит из двух уровней: уровень конфигурации проекта и уровень приложений. Такое разделение позволяет вынести общие настройки в отдельную директорию, а бизнес-логику сгруппировать внутри приложений. При масштабировании системы новые функции добавляются в виде новых при-

ложений, не затрагивая ядро проекта.

1) Корневая директория проекта.

Корневая директория `agro_спес` содержит два основных элемента: управляющий скрипт `manage.py` и файл базы данных `db.sqlite3`.

Скрипт `manage.py` является точкой входа для всех команд управления проектом. Через него выполняются создание миграций, запуск сервера разработки, создание суперпользователя и другие административные действия. Данный файл создаётся Django автоматически и не требовал изменений в ходе работы.

Файл `db.sqlite3` представляет собой базу данных SQLite. Он появляется после первого применения миграций и содержит все таблицы с данными. В условиях учебного проекта этого файла достаточно для полноценной работы. При переносе проекта на другой компьютер достаточно скопировать всю директорию, и база данных будет доступна без дополнительных настроек.

2) Директория `config`.

Директория `config` содержит настройки проекта. Она создаётся автоматически при выполнении команды `startproject`. В рамках работы были задействованы два файла из этой директории.

Файл `settings.py` содержит все настройки Django: подключённые приложения, параметры базы данных, настройки статических файлов и другие конфигурационные параметры. В ходе работы в нём было изменено только одно: в список `INSTALLED_APPS` добавлено созданное приложение `warehouse`, чтобы Django знал о его существовании и мог работать с его моделями.

Файл `urls.py` уровня проекта определяет соответствие между URL-адресами и обработчиками. В стандартную конфигурацию была добавлена ссылка на URL-конфигурацию приложения `warehouse` с помощью функции `include`. Также оставлен стандартный маршрут к административной панели.

3) Директория `warehouse`.

Директория `warehouse` является основным приложением проекта. Она содержит весь программный код, разработанный автором. Приложение создано командой `startapp`, после чего его файлы были наполнены конкретной логикой.

Файл `models.py` содержит описание моделей базы данных. Эта часть проекта была подробно рассмотрена в параграфе 2.2. Здесь определены три класса: `Worker` (работник), `SpecOdejda` (спецодежда) и `IssuedItem` (запись о выдаче).

Файл `views.py` содержит функции-представления, обрабатывающие запросы пользователя. В нём реализовано шесть функций: `dashboard` (главная страница), `worker_list` (список работников), `stock_list` (складские остатки), `issue_form` (форма выдачи), `run_simulation` (имитация износа) и `worn_items_list` (список изношенной одежды). Подробный разбор ключевых функций будет дан в следующем параграфе.

Файл `urls.py` уровня приложения был создан автором вручную. Он связывает адреса страниц с конкретными функциями из `views.py`. Каждому маршруту присвоено уникальное имя, используемое в шаблонах для генерации ссылок.

Файл `admin.py` содержит настройки встроенной административной панели Django. В нём зарегистрированы все три модели с указанием полей для отображения и фильтрации. Это позволило использовать админ-панель для быстрого наполнения базы тестовыми данными без создания отдельных форм.

4) Директория `templates`.

Директория `templates` содержит HTML-шаблоны страниц. Django требует размещать шаблоны внутри поддиректории с именем приложения, поэтому полный путь выглядит как `templates/warehouse/`.

Базовый шаблон `base.html` определяет общую структуру всех страниц: заголовок, навигационную панель и основную область содержимого. В нём также подключается CSS-фреймворк Bootstrap 5 через CDN. Все остальные шаблоны наследуются от базового с помощью тега `extends` и переопределяют только блок `content`. Такой подход исключает дублирование HTML-кода и обеспечивает единый стиль оформления.

Остальные пять шаблонов соответствуют страницам системы: `dashboard.html` (главная панель), `worker_list.html` (список работников), `stock_list.html` (склад), `issue_form.html` (форма выдачи) и `worn_items.html` (изношенная одежда). Каждый из них будет подробно описан в параграфе, посвящённом веб-интерфейсу.

5) Файлы, оставленные без изменений.

Ряд файлов, создаваемых Django автоматически, не потребовал модификации. Файл `__init__.py` делает директорию пакетом Python. Файл `apps.py` содержит конфигурацию приложения. Файл `wsgi.py` отвечает за взаимодействие с веб-сервером и актуален только при развёртывании на боевом сервере. Эти файлы сохранены в исходном виде.

Для наглядности полная структура проекта с комментариями представлена ниже (Рис. 9):

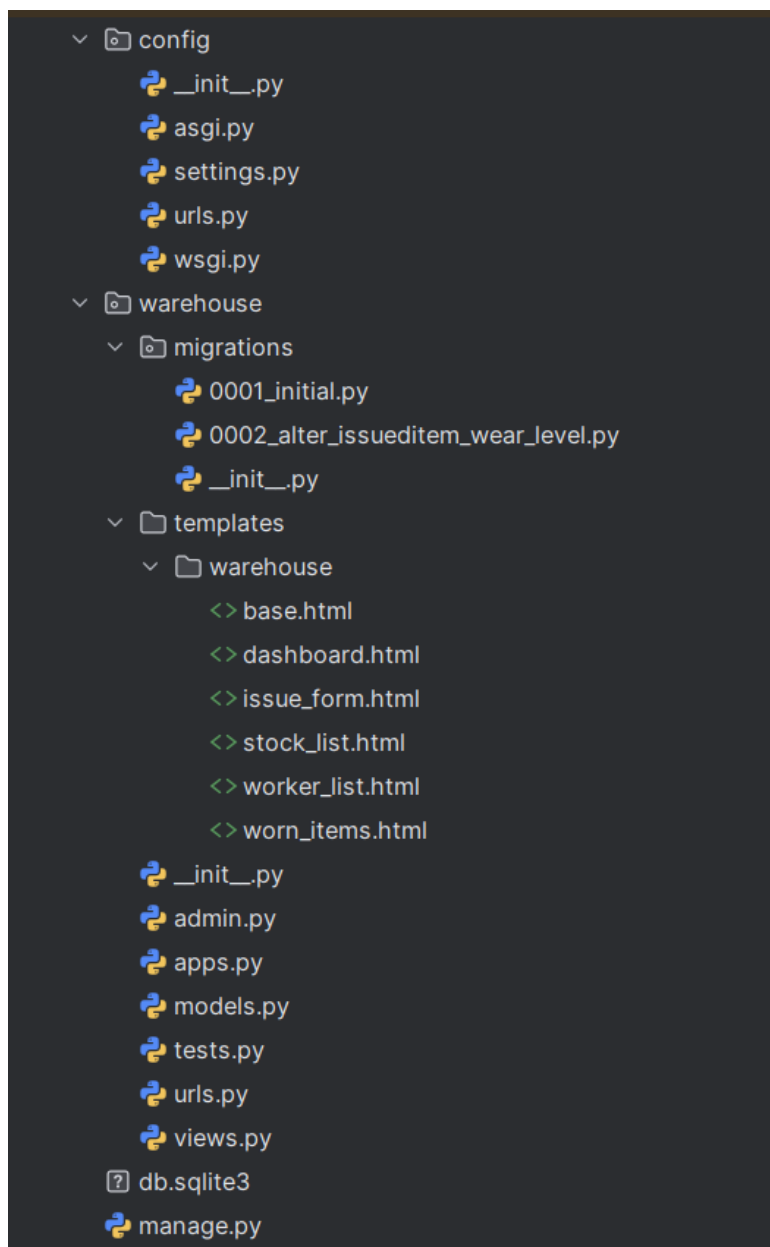


Рисунок 9 — Структура проекта

Таким образом, структура проекта соответствует стандартам Django-разработки. Все компоненты, отвечающие за бизнес-логику, сосредоточены в приложении warehouse. Настройки проекта вынесены в директорию config. Такая организация упрощает навигацию по коду и делает проект понятным для разработчика, впервые знакомящегося с системой.

2.5. Разработка веб-интерфейса информационной системы

Веб-интерфейс системы построен на основе шаблонов Django с использованием CSS-фреймворка Bootstrap 5. Все страницы наследуются от базового шаблона `base.html`, который содержит навигационную панель и подключает Bootstrap через CDN. Это обеспечивает единый стиль оформления и упрощает добавление новых страниц.

Навигационная панель содержит ссылки на все основные разделы системы: главная страница, список работников, склад, форма выдачи, запуск имитации и список изношенной одежды. Панель закреплена в верхней части страницы и доступна с любого раздела.

Главная страница (Dashboard) (Рис. 10) выполняет роль информационной панели, отображающей текущее состояние системы. В верхней части размещены четыре карточки с ключевыми показателями: общее количество работников, количество видов спецод одежды в номенклатуре, количество активных выдач и количество полностью изношенных предметов. Каждая карточка имеет свой цвет для визуального разделения. Ниже расположена таблица предметов, требующих замены. В неё попадают записи с износом более 50 процентов. Для наглядности степень износа отображается в виде индикатора прогресса (progress bar), цвет которого меняется в зависимости от процента: зелёный для значений до 50 процентов, жёлтый для диапазона от 50 до 80 процентов и красный для значений выше 80 процентов.

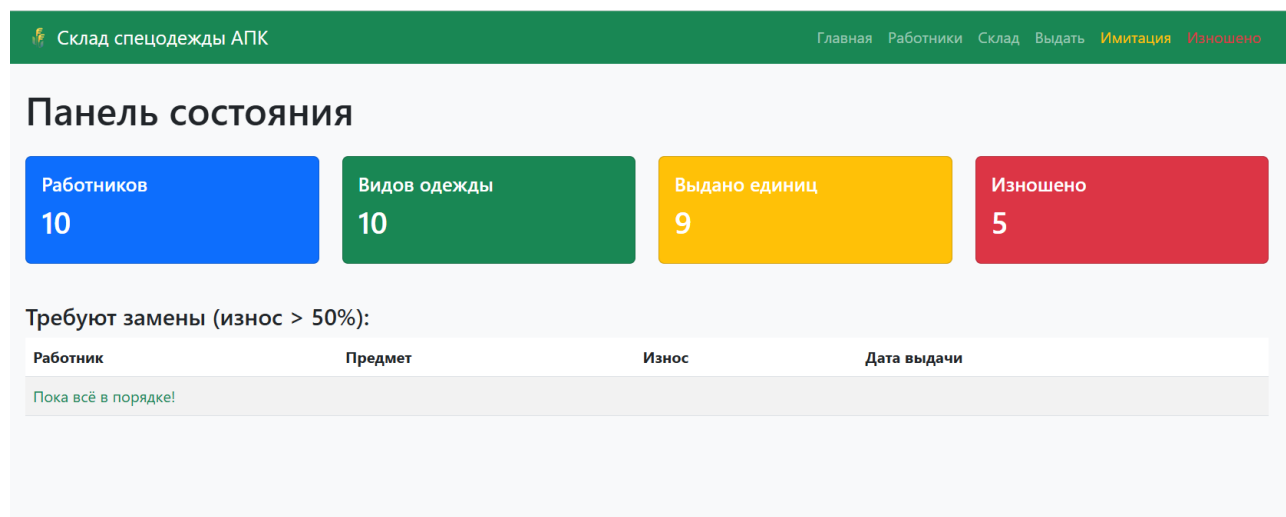


Рисунок 10 — Главная страница системы

На странице списка работников представлена таблица, каждая строка которой соответствует одному работнику. Помимо фамилии и должности, таблица содержит две информационные колонки: «Выданная одежда» и «Изношенная одежда».

В первой колонке отображаются предметы, находящиеся в активном ис-

пользовании, с указанием текущего процента износа. Для новых вещей выводится пометка «новая». Во второй колонке перечислены предметы, износ которых достиг 100 процентов и которые требуют замены. Такое разделение позволяет быстро оценить, кто из работников нуждается в обновлении спецодежды.

На рисунке 11 показана страница со списком работников. Для каждого сотрудника отображается перечень выданной спецодежды с текущим уровнем износа. Предметы, полностью выработавшие свой ресурс, выделены в отдельную колонку, что позволяет оперативно определить потребность в замене.

Склад спецодежды АПК			
Главная Работники Склад Выдать Имитация Изношено			
Список работников			
ФИО	Должность	Выданная одежда	Изношенная одежда
Скворцов Фёдор Михайлович	Тракторист		Комбинезон (изношена) Перчатки (изношена)
Черняхов Петр Ильич	Тракторист		Сапоги резиновые (изношена)
Винилина Ольга Сергеевна	Доярка	Нет активной одежды	—
Воробьёва Анастасия Александровна	Доярка		Перчатки (изношена)
Сонин Андрей Семёнович	Тракторист		Комбинезон (изношена)
Иванов Сергей Петрович	Тракторист	Комбинезон рабочий летний (износ 27%) Сапоги резиновые (износ 34%) Перчатки (износ 23%)	
Кузнецова Анна Викторовна	Доярка	Комбинезон рабочий летний (износ 24%) Сапоги резиновые (износ 22%) Перчатки прорезиненные (износ 35%)	
Петров Николай Иванович	Механизатор	Комбинезон рабочий летний (износ 24%)	
Смирнова Елена Александровна	Кладовщик	Шапка зимняя (износ 36%)	
Фёдоров Андрей Васильевич	Агроном	Перчатки прорезиненные (износ 31%)	

Рисунок 11 — Раздел «Работники»

Страница склада отображает текущее состояние складских запасов. Таблица содержит наименование, тип, размер, количество на складе и нормативный срок носки для каждого вида спецодежды. В правой колонке размещён цветовой индикатор статуса: зелёный значок «В норме» для позиций с количеством более пяти единиц, жёлтый «Мало» для позиций с количеством от одной до пяти единиц и красный «Нет в наличии» для позиций с нулевым остатком. Такая индикация позволяет кладовщику мгновенно определить, какие позиции требуют пополнения.


Склад спецодежды АПК

[Главная](#)
[Работники](#)
[Склад](#)
[Выдать](#)
[Имитация](#)
[Изношено](#)

Остатки на складе

Наименование	Тип	Размер	На складе	Норматив (дней)	Статус
Комбинезон	Верхняя одежда	S	7	89	В норме
Сапоги резиновые	Обувь	42	9	180	В норме
Перчатки	Перчатки	-	7	30	В норме
Комбинезон	Верхняя одежда	M	6	90	В норме
Комбинезон рабочий летний	Верхняя одежда	52	3	90	Мало
Комбинезон рабочий летний	Верхняя одежда	48	1	90	Мало
Сапоги резиновые	Обувь	39	5	180	Мало
Перчатки прорезиненные	Перчатки	-	15	30	В норме
Куртка зимняя утеплённая	Верхняя одежда	52	0	365	Нет в наличии!
Шапка зимняя	Головной убор	-	7	180	В норме

Рисунок 12 — Раздел «Склад»

На рисунке 12 представлена страница складских остатков. Цветовая индикация в правой колонке наглядно показывает состояние запасов. Зелёный цвет означает достаточное количество, жёлтый сигнализирует о необходимости скорого пополнения, красный предупреждает о полном отсутствии данной позиции на складе.

Форма выдачи одежды позволяет кладовщику оформить выдачу спецодежды работнику. Форма содержит два выпадающих списка: выбор работника и выбор предмета одежды. Во втором списке отображаются только те позиции, которые имеются в наличии на складе, с указанием актуального остатка. При успешной выдаче на странице появляется подтверждающее сообщение, количество на складе автоматически уменьшается, а в базе создаётся новая запись с нулевым износом. В нижней части страницы отображается таблица последних десяти выдач, позволяющая отследить историю операций.

Выдать спецодежду

Работник

-- Выберите работника --

Спецодежда

-- Выберите предмет --

Выдать

Последние выдачи:

Дата	Работник	Предмет
May 25, 2026	Черняхов Петр Ильич	Сапоги резиновые
May 25, 2026	Сонин Андрей Семёнович	Комбинезон
May 25, 2026	Скворцов Фёдор Михайлович	Комбинезон
May 25, 2026	Воробьёва Анастасия Александровна	Перчатки
May 25, 2026	Скворцов Фёдор Михайлович	Перчатки
May 25, 2026	Иванов Сергей Петрович	Комбинезон рабочий летний
May 25, 2026	Иванов Сергей Петрович	Сапоги резиновые
May 25, 2026	Иванов Сергей Петрович	Перчатки

Рисунок 13 — Раздел «Выдать»

На рисунке 13 показана форма выдачи спецодежды. Интерфейс максимально упрощён: кладовщику достаточно выбрать работника и предмет из выпадающих списков и нажать кнопку. Система автоматически проверит наличие на складе, создаст запись о выдаче и скорректирует остатки.

Страница изношенной одежды содержит таблицу со всеми предметами, переведёнными в статус «Изнюшена». Для каждой записи указывается работник, его должность, наименование предмета, тип, размер и дата выдачи. В нижней части страницы выводится общее количество изношенных единиц. Эта страница служит основой для формирования заявки на закупку новой партии спец-одежды.

На рисунке 14 представлена страница со списком полностью изношенной спецодежды. Данная информация используется для планирования закупок и составления отчётности о расходовании средств индивидуальной защиты.

Список изношенной спецодежды

Работник	Должность	Предмет	Тип	Размер	Дата выдачи	Норматив (дней)
Черняхов Петр Ильич	Тракторист	Сапоги резиновые	Обувь	42	May 25, 2026	180
Сонин Андрей Семёнович	Тракторист	Комбинезон	Верхняя одежда	M	May 25, 2026	90
Скворцов Фёдор Михайлович	Тракторист	Комбинезон	Верхняя одежда	M	May 25, 2026	90
Воробьёва Анастасия Александровна	Доярка	Перчатки	Перчатки	-	May 25, 2026	30
Скворцов Фёдор Михайлович	Тракторист	Перчатки	Перчатки	-	May 25, 2026	30

Всего изношено: 5 единиц

Рисунок 14 — Раздел «Изношено»

2.6. Реализация имитационного ядра и тестирование системы

Имитационное ядро системы реализовано в виде функции `run_simulation`, вызываемой при переходе по соответствующей ссылке в навигационной панели.

Функция `run_simulation` моделирует один календарный месяц эксплуатации спецодежды. Алгоритм последовательно обрабатывает все записи со статусом «Выдана». Для каждой записи определяется нормативный срок службы `norm_days` из связанной модели `SpecOdejda`. На основе этого значения вычисляется базовый процент износа за месяц. Случайный коэффициент в диапазоне от 0,7 до 1,3 вносит элемент стохастичности, благодаря чему разные экземпляры одежды одного типа изнашиваются с разной скоростью. Это приближает модель к реальности, где износ зависит от индивидуальных условий эксплуатации.

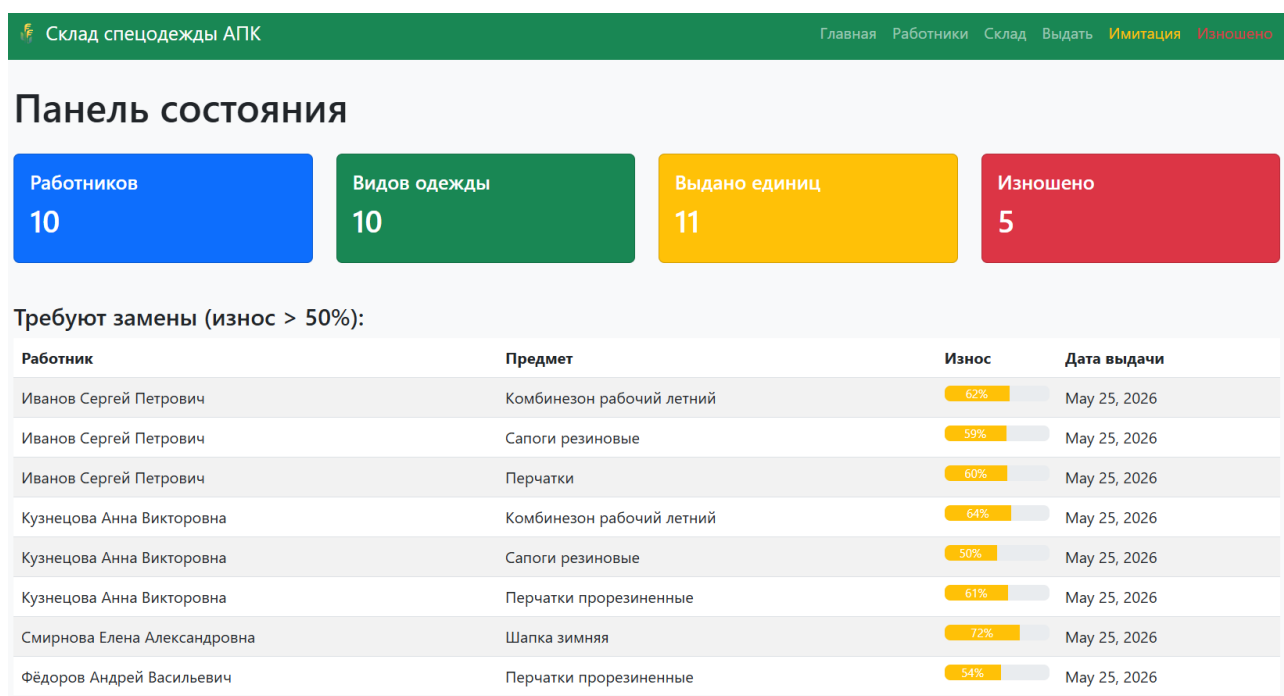


Рисунок 15 — Результат выполнения имитации

На рисунке 15 показан результат выполнения двух шагов имитации. Система выводит информационное сообщение с количеством обработанных записей и числом предметов, полностью изношенных за данный период. После выполнения имитации обновляются показатели на главной странице, отражая актуальное состояние системы.

Проведём два тестовых сценария:

- **Тестовый сценарий 1. Обычный режим эксплуатации.** Для тестирования были созданы три работника (тракторист, доярка, кладовщик) и три вида спецодежды с разными нормативными сроками: перчатки (30 дней),

комбинезон (90 дней) и сапоги (180 дней). Каждому работнику выдан полный комплект. После первого шага имитации износ перчаток составил от 70 до 100 процентов в зависимости от случайного коэффициента. Комбинезоны показали износ в диапазоне от 23 до 39 процентов, сапоги от 12 до 22 процентов. Это соответствует ожидаемому поведению: предметы с меньшим нормативным сроком изнашиваются быстрее.

- **Тестовый сценарий 2. Длительная эксплуатация.** После пяти последовательных шагов имитации все перчатки перешли в статус «Изношена», комбинезоны достигли износа в диапазоне от 80 до 100 процентов, а сапоги сохранили 40-70 процентов ресурса. Система корректно отобразила изношенные предметы на соответствующей странице и в таблице работников.

На рисунке 16 показана страница изношенной одежды после проведения пяти шагов имитации. Как видно из таблицы, предметы с коротким нормативным сроком (перчатки) изнашивались полностью, в то время как сапоги всё ещё сохраняют остаточный ресурс. Это подтверждает корректность работы имитационного алгоритма.

Ковалева Ирина Игоревна	Доярка	Сапоги резиновые (износ 77%) Комбинезон рабочий летний (износ 92%)	Перчатки прорезиненные (изношена)
Алтухов Виктор Аркадьевич	Тракторист	Комбинезон рабочий летний (износ 95%) Сапоги резиновые (износ 67%)	Перчатки прорезиненные (изношена)
Бунин Александр Артёмович	Кладовщик	Комбинезон рабочий летний (износ 95%) Сапоги резиновые (износ 71%)	Перчатки прорезиненные (изношена)

Рисунок 16 — Состояние спецодежды после проведения имитаций

Проведённое тестирование подтвердило, что разработанная система корректно выполняет все заявленные функции. Учёт выдачи и складских остатков работает без ошибок, имитационное моделирование отражает процесс износа с учётом нормативных сроков и случайных факторов. Система позволяет в ускоренном режиме «прожить» несколько месяцев эксплуатации и получить прогноз потребности в замене спецодежды.

Заключение

В ходе выполнения курсовой работы была спроектирована и разработана информационная система для имитационного моделирования логистических процессов обеспечения спецодеждой в агропромышленном комплексе. Достижение поставленной цели потребовало последовательного решения ряда теоретических и практических задач.

В теоретической части работы были изучены основы имитационного моделирования как метода исследования сложных систем. Установлено, что имитационный подход особенно эффективен в ситуациях, когда проведение реальных экспериментов невозможно или экономически нецелесообразно. Применительно к учёту спецодежды это означает возможность «имитировать» несколько месяцев эксплуатации за секунды и получить прогноз потребностей без остановки работы предприятия.

Проведённый анализ трёх основных подходов к имитационному моделированию (системная динамика, дискретно-событийный подход и агентное моделирование) показал, что для задачи учёта спецодежды наиболее подходящим является сочетание двух последних. В разработанной системе единицы выданной одежды выступают в роли агентов с индивидуальными параметрами, а время изменяется дискретно, шагами по одному месяцу.

Рассмотрение особенностей логистических процессов в агропромышленном комплексе выявило факторы, которые необходимо учитывать при проектировании системы: многообразие профессий и номенклатуры спецодежды, существенные различия в нормативных сроках службы разных предметов, индивидуальный характер износа у разных работников и жёсткое нормативное регулирование со стороны государства.

На этапе проектирования были определены три основные сущности базы данных (работник, спецодежда, запись о выдаче) и связи между ними. Разработан алгоритм имитационного моделирования, основанный на нормативных сроках службы и включающий случайный фактор для приближения модели к реальным условиям эксплуатации.

Практическая значимость разработанной системы заключается в возможности её использования на малых и средних сельскохозяйственных предприятиях. Система не требует специального обучения персонала, функционирует через стандартный браузер и легко адаптируется под конкретную номенклатуру изделий. Внедрение системы позволит кладовщику не просто фиксировать текущие остатки, но и прогнозировать будущие потребности, своевременно формируя заявки на закупку.

К возможным направлениям дальнейшего развития системы можно отнести: добавление ролевой модели (разграничение прав кладовщика и руководителя), реализацию автоматической отправки уведомлений о приближающемся износе на электронную почту, добавление возможности выбора режима интен-

сивности эксплуатации, а также формирование отчётов и графиков потребностей на основе накопленных данных имитации.

Таким образом, все поставленные задачи решены, цель курсовой работы достигнута. Разработанная информационная система демонстрирует применимость методов имитационного моделирования к задачам управления логистическими процессами в агропромышленном комплексе и может служить основой для создания более сложных решений в данной предметной области.

Список использованных источников

1. Анфилатов В.С., Емельянов А.А., Кукушкин А.А. Системный анализ в управлении: учебное пособие. – М.: Финансы и статистика, 2018. – 368 с.
2. Гагарина Л.Г. Разработка и эксплуатация автоматизированных информационных систем: учебное пособие. – М.: Форум, 2019. – 384 с.
3. Дронов В.А. Django 3.0. Практика создания веб-сайтов на Python. – СПб.: БХВ-Петербург, 2021. – 672 с.
4. Карпов Ю.Г. Имитационное моделирование систем. Введение в моделирование с AnyLogic: учебное пособие. – СПб.: БХВ-Петербург, 2020. – 400 с.
5. Кузнецов С.Д. Основы баз данных [Электронный ресурс]. – URL: <http://citforum.ru/database/osbd/> (Дата обращения: 22.05.2026).
6. Мезенцев К.Н. Моделирование систем в среде Python: учебное пособие. – М.: КноРус, 2020. – 256 с.
7. Официальная документация Django [Электронный ресурс]. – URL: <https://docs.djangoproject.com/> (Дата обращения: 24.05.2026).
8. Официальный сайт Python [Электронный ресурс]. – URL: <https://docs.python.org/3/> (Дата обращения: 24.05.2026).
9. Приказ Министерства труда и социальной защиты РФ от 09.12.2014 № 997н «Об утверждении Типовых норм бесплатной выдачи специальной одежды, специальной обуви и других средств индивидуальной защиты» [Электронный ресурс]. – URL: <https://docs.cntd.ru/document/420242640> (Дата обращения: 16.05.2026).
10. Росстат. Сельское хозяйство, охота и лесное хозяйство в России. 2023: стат. сб. – М., 2024. – 245 с.
11. Советов Б.Я., Яковлев С.А. Моделирование систем: учебник. – М.: Юрайт, 2019. – 344 с.
12. Федеральный закон от 28.12.2013 № 426-ФЗ «О специальной оценке условий труда» [Электронный ресурс]. – URL: http://www.consultant.ru/document/cons_doc_LAW_156555/ (Дата обращения: 16.05.2026).
13. Шарден Б., Массарон Л., Боскетти А. Крупномасштабное машинное обучение вместе с Python. – М.: ДМК Пресс, 2018. – 358 с.

Приложения

Приложение А. Листинги программного кода

Листинг А.1. Файл warehouse/models.py

```
1      from django.db import models
2
3      class Worker(models.Model):
4          """Работник доярка(, тракториститд ..)"""
5          full_name = models.CharField(max_length=100,
6                                       verbose_name="ФИО")
7          position = models.CharField(max_length=100,
8                                      verbose_name="Должность")
9
10         def __str__(self):
11             return f"{self.full_name} ({self.position})"
12
13         class Meta:
14             verbose_name = "Работник"
15             verbose_name_plural = "Работники"
16
17         class SpecOdejda(models.Model):
18             """Номенклатура спецодеждынаскладе """
19             TYPES = [
20                 ('upper', 'Верхняя одежда'),
21                 ('shoes', 'Обувь'),
22                 ('gloves', 'Перчатки'),
23                 ('head', 'Головной убор'),
24                 ('other', 'Прочее'),
25             ]
26
27             name = models.CharField(max_length=150,
28                                    verbose_name="Наименование")
29             type = models.CharField(max_length=20, choices=
30                                    TYPES, verbose_name="Тип")
31             size = models.CharField(max_length=10, verbose_name=
32                                    "Размер")
33             norm_days = models.IntegerField(verbose_name="Срок
34                                             носкипоноормативудней ()")
35             quantity_in_stock = models.IntegerField(default=0,
36                                                      verbose_name="Количество наскладе ")
37
38             def __str__(self):
39                 return f"{self.name} разм(. {self.size})"
40
41             class Meta:
42                 verbose_name = "Единица спецодежды"
43                 verbose_name_plural = "Спецодежда"
```

```

37
38 class IssuedItem(models.Model):
39     """Журнал выдачи:
        какая вещь кому выдана и её текущее состояние """
40     STATUSES = [
41         ('issued', 'Выдана'),
42         ('worn_out', 'Изношена'),
43         ('written_off', 'Списана'),
44     ]
45
46     worker = models.ForeignKey(Worker, on_delete=models
        .CASCADE, verbose_name="Работник")
47     spec_odejda = models.ForeignKey(SpecOdejda,
        on_delete=models.CASCADE, verbose_name="
        Спецодежда")
48     issue_date = models.DateField(auto_now_add=True,
        verbose_name="Дата выдачи")
49     wear_level = models.IntegerField(default=0,
        verbose_name="Износ (%)")
50     status = models.CharField(max_length=20, choices=
        STATUSES, default='issued', verbose_name="
        Статус")
51
52     def __str__(self):
53         return f"{self.spec_odejda.name} → {self.worker.
            full_name}"
54
55     class Meta:
56         verbose_name = "Запись о выдаче "
57         verbose_name_plural = "Журнал выдачи"

```

Листинг А.2. Файл warehouse/views.py

```

1         from django.shortcuts import render, redirect
2         from django.contrib import messages
3         from .models import Worker, SpecOdejda, IssuedItem
4
5         def dashboard(request):
6             """Главная страница с общей статистикой """
7             workers_count = Worker.objects.count()
8             items_count = SpecOdejda.objects.count()
9             issued_count = IssuedItem.objects.filter(status='
                issued').count()
10            worn_count = IssuedItem.objects.filter(status='
                worn_out').count()
11            worn_items = IssuedItem.objects.filter(
                wear_level__gte=50, status='issued')
12
13            context = {

```

```

14         'workers_count': workers_count,
15         'items_count': items_count,
16         'issued_count': issued_count,
17         'worn_count': worn_count,
18         'worn_items': worn_items,
19     }
20     return render(request, 'warehouse/dashboard.html',
21                   context)
22
23     def worker_list(request):
24         """Список всех работников """
25         workers = Worker.objects.all()
26         return render(request, 'warehouse/worker_list.html',
27                       , {'workers': workers})
28
29     def stock_list(request):
30         """Остатки на складе """
31         stock = SpecOdejda.objects.all()
32         return render(request, 'warehouse/stock_list.html',
33                       {'stock': stock})
34
35     def issue_form(request):
36         """Форма выдачи одежды работнику """
37         if request.method == 'POST':
38             worker_id = request.POST.get('worker_id')
39             item_id = request.POST.get('item_id')
40             worker = Worker.objects.get(id=worker_id)
41             item = SpecOdejda.objects.get(id=item_id)
42
43             if item.quantity_in_stock > 0:
44                 IssuedItem.objects.create(
45                     worker=worker,
46                     spec_odejda=item,
47                     wear_level=0,
48                 )
49                 item.quantity_in_stock -= 1
50                 item.save()
51                 messages.success(request, f'{item.name}
52                                     выдан работнику {worker.full_name}')
53             else:
54                 messages.error(request, f'{item.name}
55                                 отсутствует на складе !')
56             return redirect('issue_form')
57
58     workers = Worker.objects.all()
59     items = SpecOdejda.objects.filter(
60         quantity_in_stock__gt=0)
61     recent_issues = IssuedItem.objects.all().order_by('
62                 -issue_date')[:10]
63
64     context = {

```

```

58         'workers': workers,
59         'items': items,
60         'recent_issues': recent_issues,
61     }
62     return render(request, 'warehouse/issue_form.html',
63                     context)
64
65     def run_simulation(request):
66         """Имитация износа за один месяц """
67         import random
68         issued_items = IssuedItem.objects.filter(status='
69             issued')
70         count_processed = 0
71         count_worn_out = 0
72
73         for item in issued_items:
74             norm_days = item.spec_odejda.norm_days
75             if norm_days <= 0:
76                 norm_days = 30
77             base_wear = (30 / norm_days) * 100
78             random_factor = random.uniform(0.7, 1.3)
79             actual_wear = round(base_wear * random_factor, 1)
80
81             item.wear_level += actual_wear
82             if item.wear_level >= 100:
83                 item.wear_level = 100
84                 item.status = 'worn_out'
85                 count_worn_out += 1
86                 item.save()
87                 count_processed += 1
88
89             messages.success(request, f'Имитация
90                 месяца завершена ! Обработано {count_processed}
91                 вещей.')
92             return redirect('dashboard')
93
94     def worn_items_list(request):
95         """Список изношенной одежды """
96         worn_items = IssuedItem.objects.filter(status='
97             worn_out').order_by('-issue_date')
98         return render(request, 'warehouse/worn_items.html',
99                         {'worn_items': worn_items})

```

Листинг А.3. Файл warehouse/urls.py

```

1     from django.urls import path
2     from . import views
3
4     urlpatterns = [

```



```

5         path('', views.dashboard, name='dashboard'),
6         path('workers/', views.worker_list, name='
            worker_list'),
7         path('stock/', views.stock_list, name='stock_list')
            ,
8         path('issue/', views.issue_form, name='issue_form')
            ,
9         path('simulate/', views.run_simulation, name='
            run_simulation'),
10        path('worn/', views.worn_items_list, name='
            worn_items'),
11    ]

```

Листинг А.4. Файл warehouse/admin.py

```

1         from django.contrib import admin
2         from .models import Worker, SpecOdejda, IssuedItem
3
4         @admin.register(Worker)
5         class WorkerAdmin(admin.ModelAdmin):
6             list_display = ('full_name', 'position')
7             search_fields = ('full_name',)
8
9         @admin.register(SpecOdejda)
10        class SpecOdejdaAdmin(admin.ModelAdmin):
11            list_display = ('name', 'type', 'size', '
                quantity_in_stock', 'norm_days')
12            list_filter = ('type',)
13
14        @admin.register(IssuedItem)
15        class IssuedItemAdmin(admin.ModelAdmin):
16            list_display = ('worker', 'spec_odejda', '
                issue_date', 'wear_level', 'status')
17            list_filter = ('status',)

```

Приложение Б. Скриншоты работы системы

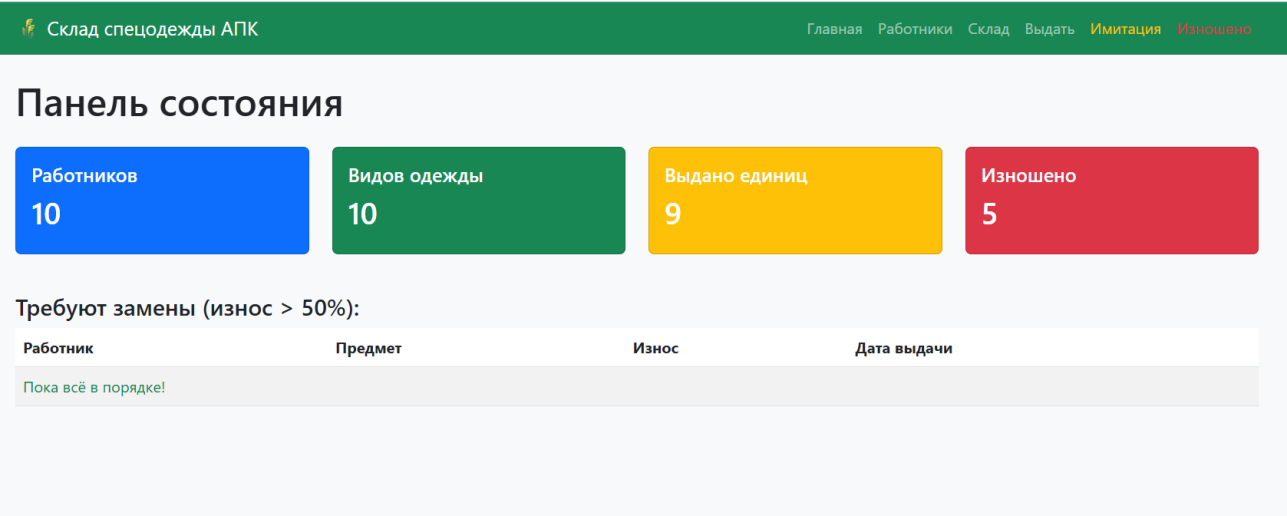


Рисунок Б.1 — Главная страница системы

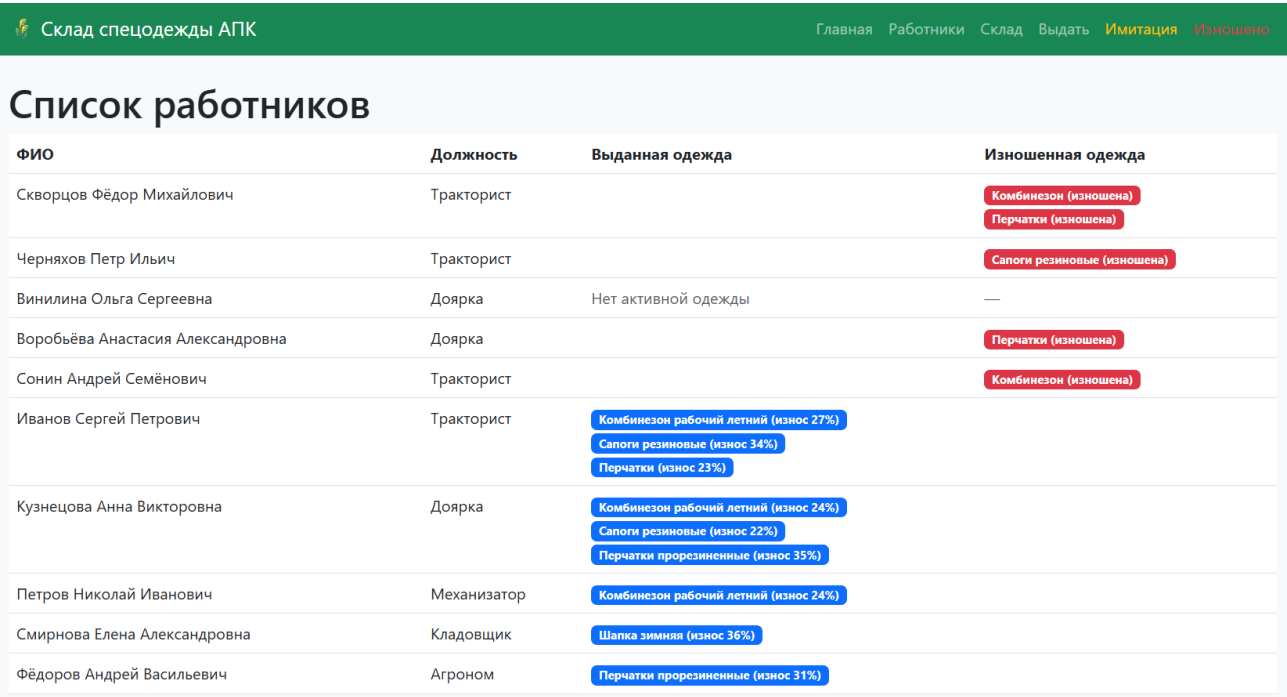


Рисунок Б.2 — Страница списка работников

🔍 Склад спецодежды АПК

ГлавнаяРаботникиСкладВыдатьИмитацияИзношено

Остатки на складе

Наименование	Тип	Размер	На складе	Норматив (дней)	Статус
Комбинезон	Верхняя одежда	S	7	89	В норме
Сапоги резиновые	Обувь	42	9	180	В норме
Перчатки	Перчатки	-	7	30	В норме
Комбинезон	Верхняя одежда	M	6	90	В норме
Комбинезон рабочий летний	Верхняя одежда	52	3	90	Мало
Комбинезон рабочий летний	Верхняя одежда	48	1	90	Мало
Сапоги резиновые	Обувь	39	5	180	Мало
Перчатки прорезиненные	Перчатки	-	15	30	В норме
Куртка зимняя утеплённая	Верхняя одежда	52	0	365	Нет в наличии!
Шапка зимняя	Головной убор	-	7	180	В норме

Рисунок Б.3 — Страница складских остатков

🔍 Склад спецодежды АПК

ГлавнаяРаботникиСкладВыдатьИмитацияИзношено

Выдать спецодежду

Работник

-- Выберите работника --

Спецодежда

-- Выберите предмет --

Выдать

Последние выдачи:

Дата	Работник	Предмет
May 25, 2026	Черняхов Петр Ильич	Сапоги резиновые
May 25, 2026	Сонин Андрей Семёнович	Комбинезон
May 25, 2026	Скворцов Фёдор Михайлович	Комбинезон
May 25, 2026	Воробьёва Анастасия Александровна	Перчатки
May 25, 2026	Скворцов Фёдор Михайлович	Перчатки
May 25, 2026	Иванов Сергей Петрович	Комбинезон рабочий летний
May 25, 2026	Иванов Сергей Петрович	Сапоги резиновые
May 25, 2026	Иванов Сергей Петрович	Перчатки

Рисунок Б.4 — Форма выдачи спецодежды

Список изношенной спецодежды

Работник	Должность	Предмет	Тип	Размер	Дата выдачи	Норматив (дней)
Черняхов Петр Ильич	Тракторист	Сапоги резиновые	Обувь	42	May 25, 2026	180
Сонин Андрей Семёнович	Тракторист	Комбинезон	Верхняя одежда	M	May 25, 2026	90
Скворцов Фёдор Михайлович	Тракторист	Комбинезон	Верхняя одежда	M	May 25, 2026	90
Воробьева Анастасия Александровна	Доярка	Перчатки	Перчатки	-	May 25, 2026	30
Скворцов Фёдор Михайлович	Тракторист	Перчатки	Перчатки	-	May 25, 2026	30

Всего изношено: 5 единиц

Рисунок Б.5 — Страница изношенной одежды

Панель состояния

Работников 10	Видов одежды 10	Выдано единиц 11	Изношено 5
------------------	--------------------	---------------------	---------------

Требуют замены (износ > 50%):

Работник	Предмет	Износ	Дата выдачи
Иванов Сергей Петрович	Комбинезон рабочий летний	62%	May 25, 2026
Иванов Сергей Петрович	Сапоги резиновые	59%	May 25, 2026
Иванов Сергей Петрович	Перчатки	60%	May 25, 2026
Кузнецова Анна Викторовна	Комбинезон рабочий летний	64%	May 25, 2026
Кузнецова Анна Викторовна	Сапоги резиновые	50%	May 25, 2026
Кузнецова Анна Викторовна	Перчатки прорезиненные	61%	May 25, 2026
Смирнова Елена Александровна	Шапка зимняя	72%	May 25, 2026
Фёдоров Андрей Васильевич	Перчатки прорезиненные	54%	May 25, 2026

Рисунок Б.6 — Административная панель Django